



به نام خدا



آزمایشگاه سیستم‌عامل - بهار 1405

پروژه اول: آشنایی با هسته سیستم‌عامل xv6

طراحان: بهراد علمی، کسری کاشانی، البرز محمودیان

مقدمه

سیستم‌عامل xv6 یک سیستم‌عامل آموزشی است که در سال 2006 توسط محققان دانشگاه MIT طرح و توسعه داده شده است. این سیستم‌عامل عموماً به زبان C و با استفاده از هسته Unix Version 6 نوشته شده و روی معماری x86 قابل اجرا می‌باشد. سیستم‌عامل xv6 علی‌رغم سادگی و حجم کم، دربردارنده نکات حائز اهمیت در طراحی بوده و برای مقاصد آموزشی بسیار پرکاربرد است. سابقاً، در درس سیستم‌عامل دانشگاه تهران از هسته سیستم‌عامل لینوکس استفاده شده که به دلیل پیچیدگی بالا این تغییر (به xv6) اتفاق افتاده است. در این پروژه، ضمن آشنایی با معماری و برخی نکات پیاده‌سازی این سیستم‌عامل، آن را اجرا و اشکال‌زدایی خواهیم کرد و همچنین برنامه‌ای در سطح کاربر نوشته خواهد شد که روی آن قابل اجرا باشد.

آشنایی با سیستم‌عامل و xv6

کدهای مربوط به سیستم‌عامل xv6 از لینک زیر قابل دسترسی است:

<https://github.com/mit-pdos/xv6-public>

همچنین مستندات این سیستم‌عامل در صفحه درس بارگذاری شده است. برای این پروژه، نیاز است که فصل صفر و پیوست ب از مستندات فوق را مطالعه کرده و به سؤالات داده شده پاسخ دهید. پاسخ این سوالات را در قالب یک گزارش بارگذاری خواهید کرد. همچنین توصیه می‌شود برای باب آشنایی پیوست الف را نیز مطالعه کنید.

سیستم‌عامل جزو نخستین نرم‌افزارهایی است که پس از روشن شدن سیستم، اجرا می‌گردد. این نرم‌افزار، رابط نرم‌افزارهای کاربردی¹ با سخت‌افزار رایانه است.

(1) سه وظیفه اصلی سیستم‌عامل را نام ببرید.

¹ Application programming interface (API)

- (2) آیا وجود سیستم عامل در تمام دستگاه ها الزامی است؟ چرا؟ در چه شرایطی استفاده از سیستم عامل لازم است؟
- (3) معماری سیستم عامل xv6 چیست؟ چه دلایلی در دفاع از نظر خود دارید؟
- (4) سیستم عامل xv6 یک سیستم تک وظیفه‌ای² است یا چندوظیفه‌ای³؟
- (5) همانطور که می‌دانید به طور کلی چندوظیفگی تعمیمی است از حالت چندبرنامگی⁴. چه تفاوتی میان یک برنامه⁵ و یک پردازنده⁶ وجود دارد؟
- (6) ساختار یک پردازنده در سیستم عامل xv6 از چه بخش‌هایی تشکیل شده است؟ این سیستم عامل به طور کلی چگونه پردازنده را به پردازنده‌های مختلف اختصاص می‌دهد؟
- (7) مفهوم file descriptor در سیستم عامل‌های مبتنی بر UNIX چیست؟ عملکرد pipe در سیستم عامل xv6 چگونه است و به طور معمول برای چه هدفی استفاده می‌شود؟
- (8) فراخوانی‌های سیستمی exec و fork در سیستم عامل xv6 چه عملیاتی را انجام می‌دهند؟ از نظر طراحی، ادغام نکردن این دو چه مزیتی دارد؟

اجرا و برپایی⁷ سیستم عامل xv6 روی شبیه ساز Qemu

در این بخش به اجرای سیستم عامل xv6 خواهیم پرداخت. علی‌رغم اینکه این سیستم عامل قابلیت اجرای مستقیم روی سخت‌افزار را دارد، به دلیل آسیب‌پذیری بالا و رعایت مسائل ایمنی از این کار اجتناب نموده و سیستم عامل را به کمک برابرساز⁸ Qemu روی سیستم عامل لینوکس اجرا خواهیم کرد. برای این منظور لازم است که کدهای مربوط به سیستم عامل xv6 را از لینک ارائه شده clone و یا دانلود کنیم.

یکی از روش‌های متداول کامپایل و ایجاد نرم‌افزارهای بزرگ استفاده از ابزار یا سامانه ساخت⁹ Makefile است. این ابزار با فراهم کردن بستری برای پردازش فایل‌های موجود در کد منبع برنامه، در قالب قوانین¹⁰، دستورالعمل‌ها¹¹ و متغیرها¹²، به توسعه دهنده امکان مشخص کردن نحوه کامپایل و لینک فایل‌های دودویی به یکدیگر را می‌دهد. در xv6 تنها یک Makefile وجود داشته و تمامی فایل‌های سیستم عامل نیز در یک پوشه قرار دارند.

² Single-tasking system

³ Multitasking system

⁴ Multiprogramming

⁵ Program

⁶ Process

⁷ Setup

⁸ Emulator

⁹ Build system

¹⁰ Rules

¹¹ Recipes

¹² Variables

در ادامه با اجرای دستور make در پوشه دانلود، سیستم عامل کامپایل می شود و در نهایت با اجرای دستور make qemu سیستم عامل روی برابرساز اجرا خواهد شد (توجه شود که فرض شده Qemu از قبل بر روی سیستم عامل شما نصب بوده است. در غیر این صورت ابتدا آن را نصب نمایید).

9) دستور make -n را اجرا نمایید. کدام دستور، فایل نهایی هسته را می سازد؟

اضافه کردن یک متن به Boot Message

کدهای خام را به نحوی تغییر دهید که پس از بوت شدن سیستم عامل روی ماشین مجازی Qemu، نام اعضای گروه نمایش داده شود. تصویر نتیجه را نیز در گزارش کار خود قرار دهید.

اضافه کردن قابلیت های جدید به کنسول

در این بخش، چند قابلیت جدید و پرکاربرد را به کنسول خود اضافه خواهید کرد:

1. قابلیت جابه جایی مکان نما¹³: به کمک این قابلیت کاربر می تواند:

- با فشردن کلیدهای → و ← مکان نما را بین کاراکترهای موجود در رشته ورودی کنونی کنسول، به چپ یا راست ببرد.
- با فشردن Ctrl+D، مکان نما در ابتدای (اولین کاراکتر از) کلمه بعد (در صورت وجود) قرار بگیرد.
- با فشردن Ctrl+A، مکان نما در ابتدای کلمه فعلی قرار بگیرد. در صورتی که مکان نما قبل از فشردن Ctrl+A، در ابتدای کلمه فعلی یا روی یک space بود، به ابتدای کلمه قبل (در صورت وجود) منتقل شود. دقت کنید که هر کلمه با یک یا چند space از هم جدا می شوند.

2. قابلیت حذف آخرین کاراکتر وارد شده: کاربر با فشردن دستور Ctrl+Z می تواند آخرین کاراکتری که روی رشته ورودی کنونی کنسول چاپ شده است (از نظر زمانی) را پاک کند. دقت کنید که مکان نما هر جا که باشد، آخرین کاراکتر وارد شده پاک می شود و همچنین با چند مرتبه فشردن شدن این دستور نیز به همان تعداد کاراکتر، برعکس ترتیب زمان وارد شدن، پاک خواهند شد.

¹³ Cursor

3. قابلیت **select و paste - copy**: کاربر به کمک این قابلیت می‌تواند یک زیر رشته از رشته ورودی کنونی کنسول را انتخاب¹⁴ کند، بدین صورت که روی یک کاراکتر دستور Ctrl+S یکبار زده شود و روی یک کاراکتر دیگر نیز دستور Ctrl+S را مجدد بزند تا رشته موجود در آن بازه انتخاب شود. توجه داشته باشید که رشته انتخاب شده با رنگ سفید هایلایت شده باشد. پس از فشردن دستور Ctrl+S برای بار اول، کاربر همچنان می‌تواند در هر جایی از رشته، کاراکتر وارد یا حذف کند. در ادامه ۴ عمل زیر **رشته انتخاب شده**، قابل انجام می‌باشند:

- با یک بار فشردن کلید Backspace کل رشته پاک شود.
- با فشردن دستور Ctrl+C کل رشته صرفاً کپی شود اما رشته همچنان انتخاب شده باقی بماند.
- با فشردن دستور Ctrl+V، کل رشته با رشته از قبل کپی شده (در صورت وجود) جایگزین شود. دقت کنید که رشته از قبل کپی شده، در هر جای دیگری از ورودی (حتی بدون نیاز به انتخاب شدن) و در جایگاه مکان‌نما، با این دستور paste خواهد شد.
- با فشردن هر کدام از کلیدهای کاراکترها کل رشته پاک شود و با همان کاراکتر جدید جایگزین شود.

در پایان هر یک از این ۴ عمل به جز عمل دوم (دستور Ctrl+C)، کنسول باید به حالت عادی خود برگردد. دقت کنید که در صورتی که هر کلید یا دستور دیگری، مثل کلیدهای → و ← یا دستورات Ctrl+Z و Ctrl+S و Ctrl+A و Ctrl+D، فشرده شود، رشته‌ی select شده فقط باید deselect شود و هیچ عمل دیگری (حتی کلید یا دستور وارد شده) انجام نشود و فقط کنسول به حالت عادی خود برگردد.

4. **قابلیت شما:** به نظر شما چه قابلیت جدید و کاربردی دیگری می‌تواند به کنسول اضافه شود، به طوری که قابل پیاده‌سازی باشد؟ حداقل 2 مورد را در گزارش کار خود نام ببرید. (دقت کنید که نیازی به پیاده‌سازی آن وجود ندارد). ممکن است در ترم های آینده، از ایده های شما (به همراه اسمتان) استفاده شود!

¹⁴ Select

توجه کنید: دستورات فوق باید قابلیت اجرای ترکیبی نیز داشته باشند و عملکرد درستی از خودشان بروز دهند. هنگام تحویل، چند تست کیس ترکیبی نیز بررسی خواهند شد. علاوه بر آن، می‌بایست تست کیس‌های خود را که برای آزمون پروژه از آن استفاده می‌کنید با ذکر دلیل استفاده، در گزارش قرار دهید.

مراحل بوت سیستم عامل xv6

اجرای بوت‌لودر¹⁶

هدف از بوت، آماده‌سازی سیستم‌عامل برای سرویس‌دهی به برنامه‌های کاربر است. پس از بوت، سیستم‌عامل سازوکاری جهت ارائه سرویس به برنامه‌های کاربردی خواهد داشت که این برنامه‌ها بدون هیچ مزاحمتی بتوانند از آن استفاده نمایند. کوچکترین واحد دسترسی دیسک‌ها در رایانه‌های شخصی سکتور¹⁷ است. در اینجا هر سکتور ۵۱۲ بایت است. اگر دیسک قابل بوت باشد، نخستین سکتور آن، سکتور بوت¹⁸ نام داشته و شامل بوت‌لودر خواهد بود. بوت‌لودر کدی است که سیستم‌عامل را در حافظه بارگذاری می‌کند. یکی از روش‌های راه‌اندازی اولیه رایانه، بوت مبتنی بر سیستم ورودی/خروجی پایه¹⁹ (BIOS) است. BIOS در صورت یافتن دیسک قابل بوت، سکتور نخست آن را در آدرس 0x7C00 از حافظه فیزیکی کپی نموده و شروع به اجرای آن می‌کند.

¹⁵ Autocompletion

¹⁶ Bootloader

¹⁷ Sector

¹⁸ Boot Sector

¹⁹ Basic Input/Output System

12) در xv6 در سکتور نخست دیسک قابل بوت، محتوای چه فایلی قرار دارد؟ (راهنمایی: خروجی دستور `make -n` را بررسی نمایید.)

13) برنامه‌های کامپایل شده در قالب فایل‌های دودویی²⁰ نگه‌داری می‌شوند. فایل مربوط به بوت نیز دودویی است. نوع این فایل دودویی چیست؟ چرا از این نوع فایل دودویی استفاده شده است؟ تفاوت این نوع فایل دودویی با دیگر فایل‌های دودویی کد xv6 چیست؟ این فایل را به زبان قابل فهم انسان (اسمبلی²¹) تبدیل نمایید. (راهنمایی: از ابزار `objdump` استفاده کنید. باید بخشی از آن مشابه فایل `bootasm.S` باشد.)

14) علت استفاده از دستور `objcopy` در حین اجرای عملیات `make` چیست؟

15) در فایل‌های موجود در xv6، مشاهده می‌شود که بوت سیستم توسط فایل‌های `bootasm.S` و `bootmain.c` صورت می‌گیرد. چرا تنها از کد C استفاده نشده است؟

معماری سیستم شبیه‌سازی شده x86 است. حالت سیستم در حال اجرا در هر لحظه را به طور ساده می‌توان شامل حالت پردازنده و حافظه دانست. بخشی از حالت پردازنده در ثبات‌های آن نگه‌داری می‌شود.

16) یک ثبات عام منظوره²²، یک ثبات قطعه²³، یک ثبات وضعیت²⁴ و یک ثبات کنترلی²⁵ در معماری x86 را نام برده و وظیفه هر یک را به طور مختصر توضیح دهید.

وضعیت ثبات‌ها را می‌توان به کمک `gdb` و دستور `info registers` مشاهده نمود. وضعیت برخی از ثبات‌های دیگر نیاز به دسترسی ممتاز²⁶ دارد. به این منظور می‌توان از `qemu` استفاده نمود. کافی است با زدن `Ctrl + A` و سپس C به ترمینال `qemu` رفته و دستور `info registers` را وارد نمود. با تکرار همان دکمه‌ها می‌توان به xv6 بازگشت.

17) پردازنده‌های x86 دارای مدهای مختلفی هستند. هنگام بوت، این پردازنده‌ها در مد حقیقی²⁷ قرار داده می‌شوند؛ مدی که سیستم عامل ام‌اس‌داس²⁸ (MS DOS) در آن اجرا می‌شد. چرا؟

²⁰ Binary

²¹ Assembly

²² General Purpose Register

²³ Segment Register

²⁴ Status Registers

²⁵ Control Registers

²⁶ Privileged Access

²⁷ Real Mode

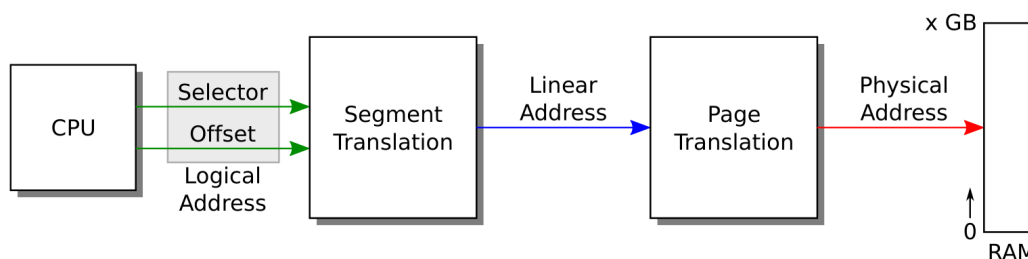
²⁸ Microsoft Disk Operating System

یک نقص اصلی این مد را بیان نمایید. آیا در پردازنده‌های دیگر مانند ARM یا RISC-V نیز مدها به همین شکل هستند یا خیر؟ توضیح دهید.

(18) یکی دیگر در از مدهای مهم، مد حفاظت‌شده²⁹ می‌باشد. وظیفه‌ی اصلی این مود چیست؟ پردازنده‌ها در چه زمانی در این مود قرار می‌گیرند؟

در ابتدا qemu یک هسته را جهت اجرای کد بوت bootasm.S فعال می‌کند. فرایند بوت در بالاترین سطح دسترسی³⁰ صورت می‌گیرد. به عبارت دیگر، بوت‌لودر امکان دسترسی به تمامی قابلیت‌های سیستم را دارد. در ادامه هسته به مد حفاظت‌شده تغییر مد می‌دهد (خط ۹۱۵۳). در مد حفاظت‌شده، آدرس مورد دسترسی در برنامه (آدرس منطقی) از طریق جداولی به آدرس فیزیکی حافظه³¹ نگاشت پیدا می‌کند.

ساختار آدرس‌دهی در این مد در شکل زیر نشان داده شده است.



(19) کد bootmain.c، هسته را با شروع از یک سکتور پس از سکتور بوت، خوانده و در آدرس 0x100000 قرار می‌دهد.³² علت انتخاب این آدرس چیست؟ چرا این آدرس از 0 شروع نشده است؟

در ادامه و در انتهای بوت، کد هسته سیستم‌عامل به طور کامل در حافظه قرار خواهد گرفت. در گام انتهایی نیز، بوت‌لودر اجرا را به هسته واگذار می‌نماید. باید کد ورود به هسته اجرا گردد. این کد اسمبلی در فایل entry.S قرار داشته و نماد (بیانگر مکانی از کد) entry از آن فراخوانی می‌گردد. آدرس این نماد در هسته بوده و حدود 0x100000 است.

²⁹ Protected Mode

³⁰ سطوح دسترسی در ادامه پروژه توضیح داده خواهد شد.

³¹ منظور از آدرس فیزیکی یک آدرس یکتا در سخت‌افزار حافظه است که پردازنده به آن دسترسی پیدا می‌کند.

³² دقت شود آدرس 0x100000 تنها برای خواندن هدر فایل elf استفاده شده است و محتوای فایل هسته در 0x100000 که توسط paddr (مخفف آدرس فیزیکی) تعیین شده است، کپی می‌شود. این آدرس در زمان لینک توسط kernel.ld تعیین شده و در فایل دودویی در قالب خاصی قرار داده شده است.

اشکال زدایی

کد هر برنامه‌ای ممکن است دارای اشکال باشد. اشکال‌زدایی ممکن است ایستا³³، پویا³⁴ و یا به صورت ترکیبی³⁵ صورت پذیرد. کشف اشکال در روش‌های ایستا، بدون اجرا و تنها بر اساس اطلاعات کد برنامه صورت می‌گیرد. به عنوان مثال کامپایلر Clang دارای تحلیل‌گرهای ایستا³⁶ برای اشکال‌زدایی اشکال‌های خاص است. اشکال‌زدایی پویا که معمولاً دقیق‌تر است، اقدام به کشف اشکال در حین اجرای برنامه می‌نماید. ابزار leak-check در ابزار Valgrind یک اشکال‌زدای پویا برای تشخیص نشتی حافظه³⁷ است. از یک منظر می‌توان اشکال‌زداهای پویا را به دو دسته تقسیم نمود: ۱) اشکال‌زداهایی که بر یک نوع اشکال خاص مانند نشتی تمرکز دارند و ۲) اشکال‌زداهایی که مستقل از نوع اشکال بوده و تنها اجرا را ردگیری³⁸ نموده و اطلاعاتی از حالت سیستم (شامل سخت‌افزار و نرم‌افزار) در حین اجرا یا پس از اجرا جهت درک بهتر رفتار برنامه برمی‌گردانند. در این بخش ابزار اشکال‌زدای گنو³⁹ (GDB)، که یک اشکال‌زدای پویا از نوع دوم است معرفی خواهد شد.

GDB یک اشکال‌زدای متداول در سیستم‌های یونیکسی بوده که در بسیاری از شرایط، نقش قابل‌توجهی در تسریع روند اشکال‌زدایی ایفا می‌کند. اشکال‌زدایی برنامه‌های تک‌ریسه‌ای⁴⁰، چندریسه‌ای⁴¹ و حتی هسته‌های سیستم‌عامل توسط این ابزار ممکن است. جهت اشکال‌زدایی xv6 با GDB، در گام نخست باید سیستم‌عامل به صورتی بوت شود که قابلیت اتصال اشکال‌زدا به آن وجود داشته باشد. مراحل اتصال به شرح زیر است:

1. در یک ترمینال دستور `make qemu-gdb` را اجرا کنید.

2. سپس در ترمینالی دیگر، فایل کد اجرایی را به عنوان ورودی به GDB بدهید.

چنانچه پیش‌تر ذکر شد کد اجرایی شامل یک نیمه هسته و یک نیمه سطح کاربر بوده که نیمه هسته، ثابت و نیمه سطح کاربر، بسته به برنامه در حال اجرا بر روی پردازنده دائماً در حال تغییر است. به این ترتیب، به عنوان مثال، هنگام اجرای برنامه cat، کدهای اجرایی سیستم شامل کد هسته و کد برنامه cat خواهند بود. جهت اشکال‌زدایی بخش سطح کاربر، کافی است دستور `gdb _cat` و جهت اشکال‌زدایی بخش هسته دستور `gdb kernel` فراخوانی شود. دقت شود در هر دو حالت، هر دو کد سطح هسته و کاربر اجرا می‌شوند. اما اشکال‌زدا فقط روی یک کد اجرایی (سطح کاربر یا هسته) کنترل داشته و تنها قادر به انجام عملیات بر روی آن قسمت خواهد بود.

³³ Static

³⁴ Dynamic

³⁵ Hybrid

³⁶ Static analyzer

³⁷ Memory Leak

³⁸ Tracing

³⁹ GNU Debugger

⁴⁰ Single-Thread

⁴¹ Multithread

3. نهایتاً با وارد کردن دستور `target remote tcp::26000` در GDB، اتصال به سیستم عامل صورت خواهد گرفت.

روند اجرای GDB

GDB می تواند در هر گام از اجرا، با ارائه حالت سیستم، به برنامه نویسی کمک کند تا حالت خطا را از حالت مورد انتظار تشخیص دهد. هنگام اجرای کد در GDB ممکن است چندین حالت رخ دهد:

1. اجرا با موفقیت جریان داشته باشد یا خاتمه یابد.
2. اجرا به علت اشکال، ناتمام مانده و برنامه متوقف شود.
3. اجرا متوقف نشده ولی حالت سیستم در برخی نقاط درونی یا در خروجی های برنامه نادرست باشد.

هدف، یافتن حالات خطای سیستم در دو وضعیت ۲ و ۳ است. به عبارتی ابتدا باید در نقطه مورد نظر، توقف صورت گرفته و سپس به کمک دستورهای حالت سیستم را استخراج نمود. برای توقف اجرا در نقاط مختلف اجرا در GDB سازوکارهای مختلفی وجود دارد:

1. در اجرای ناتمام، اجرای برنامه به طور خودکار متوقف می شود.
 2. با فشردن کلید ترکیبی `Ctrl + C` به اشکال زدا بازگشت.
- این عملیات در میان اجرا، آن را متوقف نموده و کنترل را به خط فرمان اشکال زدا منتقل می کند. مثلاً حلقه بی نهایت رخ داده باشد، می توان با این کلید ترکیبی، در نقطه ای از حلقه متوقف شد.
3. روی نقطه ای از برنامه Breakpoint قرار داد. بدین ترتیب هر رسیدن اجرا به این نقطه منجر به توقف اجرا گردد.
- روش های مختلفی برای تعیین نقطه استقرار Breakpoint وجود داشته که در این [لینک](#) قابل مشاهده است. از جمله:

انتخاب نام و شماره خط فایل

```
$ break cat.c:12
```

انتخاب نام تابع

```
$ b cat
```

انتخاب آدرس حافظه

```
$ b *0x98
```

این نقاط می توانند در سطح کاربر یا هسته سیستم عامل باشند. همچنین می توانند شرطی تعریف شوند.

4. روی خانه خاصی از حافظه Watchpoint قرار داد تا دسترسی یا تغییر مقدار آن خانه، منجر به توقف اجرا گردد.

Watchpoint ها انواع مختلفی داشته و با دستورهای خاص خود مشخص می‌گردند.

دستور زیر:

\$ watch *0x1234567

یک Watchpoint روی آدرس 0x1234567 در حافظه می‌گذارد. بدین ترتیب نوشتن در این آدرس، منجر به توقف اجرا خواهد شد.

می‌توان از نام متغیر هم استفاده نمود. مثلاً Watch v، watch را روی (آدرس) متغیر v قرار می‌دهد.

باید دقت نمود، اگر Watch روی متغیر محلی قرار داده شود، با خروج از حوزه دسترسی به آن متغیر، Watch حذف شده و به برنامه‌نویس اطلاع داده می‌شود. اگر هم آدرسی از فضای پشته⁴² داده شود، ممکن است در حین اجرا متغیرها یا داده‌های نامرتبب دیگری در آن آدرس نوشته شود. یعنی این آدرس در زمان‌های مختلف مربوط به داده‌های مختلف بوده و در عمل Watch کارایی مورد نظر را نداشته باشد.

یک مزیت مهم Watch، تشخیص وضعیت مسابقه⁴⁴ است که در فصول بعدی درس با آن آشنا خواهید شد. در این شرایط می‌توان تشخیص داد که کدام ریس⁴⁵ یا پردازنده مقدار نامناسب را در آدرس حافظه نوشته که منجر به خطا شده است.

همان‌طور که مشاهده می‌شود، خیلی از حالات با استفاده از چهار سازوکار مذکور به سهولت قابل استخراج نیستند. مثلاً حالتی که یک زنجیره خاص فراخوانی توابع وجود داشته باشد یا این که مثلاً حالتی خاص در داده‌ساختارها رخ داده و یک لیست پیوندی، چهارمین عنصرش را حذف نماید.

(20) برای مشاهده Breakpoint ها از چه دستوری استفاده می‌شود؟

(21) برای حذف یک Breakpoint از چه دستوری و چگونه استفاده می‌شود؟

کنترل روند اجرا و دسترسی به حالت سیستم

پس از توقف می‌توان با استفاده از دستورهایی به حالت سیستم دسترسی پیدا نمود. همچنین دستورهایی برای تعیین شیوه ادامه اجرا وجود دارد. در ادامه، برخی از دستورهای کنترلی و دسترسی به حالت اجرا معرفی خواهد شد.

پس از توقف روی Breakpoint می‌توان با اجرای دستورهای step و next و finish به ترتیب به دستور بعدی، به درون دستور بعدی (اگر فراخوانی تابع باشد) و به خارج از تابع کنونی (یعنی بازگشت به تابع فراخواننده) منتقل شد. به عبارت دیگر، اجرا گام‌به‌گام قابل بررسی است. بدین معنی که پیش از اجرای خط جاری برنامه سطح کاربر یا هسته، امکان دستیابی به اطلاعات متغیرها و ثبات‌ها فراهم می‌باشد.

⁴² Stack

⁴³ یعنی فضای آدرسی که داده‌هایی از جمله مقادیر متغیرهای محلی و آدرس‌های برگشت مربوط به توابع فراخوانی شده در آن قرار دارد.

⁴⁴ Race Condition

⁴⁵ Thread

به این ترتیب می‌توان برنامه را از جهت وجود حالات نادرست، بررسی نمود. همچنین دستور `continue` اجرا را تا رسیدن به نقطه توقف بعدی یا اتمام برنامه ادامه می‌دهد. با دستور `list` نیز می‌توان کد نقطه توقف را مشاهده نمود.

(22) دستور زیر را اجرا کنید. خروجی آن چه چیزی را نشان می‌دهد؟

```
$ bt
```

(23) دو تفاوت دستورهای `x` و `print` را توضیح دهید. چگونه می‌توان محتوای یک ثابت خاص را چاپ کرد؟ (راهنمایی: می‌توانید از دستور `help` استفاده نمایید: `help x` و `help print`)

(24) برای نمایش وضعیت ثابت‌ها از چه دستوری استفاده می‌شود؟ برای متغیرهای محلی چطور؟ نتیجه این دستور را در گزارش کار خود بیاورید. همچنین در گزارش خود توضیح دهید که در معماری `x86` رجیسترهای `edi` و `esi` نشانگر چه چیزی هستند؟

(25) به کمک استفاده از `GDB`، درباره ساختار `struct input` موارد زیر را توضیح دهید:

- توضیح کلی این `struct` و متغیرهای درونی آن و نقش آن‌ها.
- نحوه و زمان تغییر مقدار متغیرهای درونی (برای مثال، `input.e` در چه حالتی تغییر می‌کند و چه مقداری می‌گیرد)

اشکال زدایی در سطح کد اسمبلی

اشکال زدایی برنامه در سطوح مختلفی قابل انجام است. با توجه به این که بسیاری از جزئیات اجرا در کد سطح بالا (زبان `C`) قابل مشاهده نیست، نیاز به اشکال‌زدایی در سطح کد اسمبلی خواهد بود. به عنوان مثال بهینه‌سازی‌های ممکن است ترتیب اجرا در کد سطح بالا را تغییر داده یا بخشی از کد را حذف نماید. به عنوان مثال دیگر می‌توان از شیوه دسترسی به جداول لینکر نام برد. جزئیات دسترسی به یک تابع کتابخانه‌ای خاص یا یک متغیر سراسری آن کتابخانه دسترسی شده است، در سطح کد اسمبلی و با دسترسی به جداول لینک رخ داده و در سطح زبان سی قابل رؤیت نیست. با فشردن همزمان سه دکمه `Ctrl + X + A` رابط کاربری متنی⁴⁶ `GDB` یا همان `TUI` گشوده شده و کد اسمبلی مربوط به نقطه توقف، قابل رؤیت است. برای اطلاعات بیشتر در رابطه با این رابط کاربری می‌توانید به این [صفحه](#) مراجعه کنید.

(26) خروجی دستورات `layout src` و `layout asm` در `TUI` چیست؟

(27) برای جابه‌جایی میان توابع زنجیره فراخوانی جاری (نقطه توقف) از چه دستوراتی استفاده می‌شود؟

⁴⁶ Text user interface

دستورهای stepi و nexti معادل‌های سطح اسمبلی step و next بوده و به جای یک دستور سی، در ریزدانی یک دستورالعمل ماشین عمل می‌کنند. در شرایطی که کد مورد اشکال‌زدایی از ابتدا در زبان اسمبلی نوشته شده باشد، چاره‌ای جز استفاده از این دستورها وجود نخواهد داشت.

نکات پایانی

با توجه به کاستی‌هایی که در اشکال‌زدها وجود دارد، همچنان برخی از تکنیک‌ها در کدزنی می‌تواند بسیار راهگشا باشد. ساده‌ترین راه برای اشکال‌زدایی این است که تغییرها را اندک انجام داده و گام‌به‌گام از صحت اجرای کد، اطمینان حاصل شود. به عنوان مثال اگر آرایه‌ای ۱۰۰ عنصری تخصیص داده شده و در نقطه‌ای فراتر از مرز انتهایی آن نوشتن صورت گیرد، حافظه‌ای غیر از حافظه مربوط به آرایه دستکاری می‌گردد. چندین حالت ممکن است رخ دهد، از جمله اینکه:

1. اقدام به نوشتن در حافظه‌ای فقط خواندنی مانند کد برنامه، صورت پذیرد. در چنین شرایطی خطا رخ داده و نقطه توقف به راحتی در GDB قابل رؤیت خواهد بود.
 2. در حافظه نوشتن نامرتب نوشته شده و مشکلی پیش نیاید.
 3. در حافظه نوشتن نامرتب نوشته شود و اجرای برنامه به طرز عجیبی متوقف گردد. به طوری که GDB نقطه نامربوطی را نشان دهد. یعنی تأثیر آن بلافاصله و به طور مستقیم رخ ندهد. در چنین شرایطی استفاده ابتدایی از اشکال‌زدا راحتی راهگشا نخواهد بود. چک کردن اندازه آرایه و احتمال دسترسی به خارج آن در سطح کد، می‌توانست راحت‌تر باشد. البته در برخی موارد به سادگی و یا با تکنیک‌هایی مانند استفاده از Watch، ضبط اجرا و حرکت رو به عقب از حالت نادرست، می‌توان اشکال را یافت⁴⁷. اما تکنیک قبلی بهتر بود.
- بنابراین، استفاده از GDB در کنار دیگر ابزارها و تکنیک‌ها در پروژه‌های این درس توصیه می‌گردد. با توجه به آشنایی اولیه‌ای که با GDB فراهم شده است، می‌توان مزایای آن را برشمرد:
- اشکال‌زدایی کدهای بزرگ و کدهایی که با پیاده‌سازی آن‌ها آشنایی وجود ندارد. ممکن است نیاز باشد یک کد بزرگ را به برنامه اضافه کنید. در این شرایط اشکال‌زدایی اجرای Crash کرده در GDB درک اولیه‌ای از نقطه خرابی ارائه می‌دهد.
 - بررسی مقادیر حالت برنامه، بدون نیاز به قرار دادن دستورهای چاپ مقادیر در کد و کامپایل مجدد آن.
 - بررسی مقادیر حالت سخت‌افزار و برنامه که در سطح کد قابل رؤیت نیستند. به عنوان مثال مقدار یک اشاره‌گر به تابع، مقصد یک تابع کتابخانه‌ای، اطمینان از قرارگیری آدرس متغیر محلی در بازه حافظه پشته، این که اجرا در کدام فایل کد منبع قرار دارد، اطلاع از وضعیت فضای آدرس حین اجرا، مثلاً این که هر کتابخانه در چه آدرسی بوده و در کدام کتابخانه در حال اجرا هستیم و

⁴⁷ GDB در برنامه‌های عادی قادر به ضبط و اجرای رو به عقب برنامه است. همچنین ابزار RR که توسط شرکت موزیلا برای اشکال‌زدایی فایرفاکس ارائه شده است امکان انجام همین عملیات را به صورت قطعی دارد. این قطعیت، در اشکال‌زدایی کدهای همروند و وضعیت مسابقه بسیار کمک‌کننده است.

- تشخیص اشکال‌های پیچیده مانند این که کدام ریشه، یک متغیر را دستکاری نموده یا چرا یک متغیر مقدار نادرستی داشته یا مقداردهی اولیه نشده است. این اشکال‌های با کمک Watch و ضبط و اجرای مجدد رو به جلو/عقب به راحتی قابل تشخیص هستند.

- به تمامی سوالات پاسخ داده و پاسخ‌ها و نتایج را در گزارش کار خود قید کنید.
- بخش خوبی از نمره شما را پاسخ دهی به سوالات مطرح شده تشکیل می‌دهد که به شما در درک نحوه کارکرد xv6 کمک می‌کند.
- سوالات را به طور خلاصه پاسخ دهید.
- در پیاده‌سازی خود در خصوص مواردی که ذکر نشده‌اند می‌توانید فرضی منطقی و بر اساس کارکرد پوسته⁴⁹ لینوکس خود در نظر بگیرید.
- مطالعه فصل صفر و پیوست ب از کتاب xv6 الزامی بوده مطالعه پیوست الف نیز توصیه می‌شود.

موفق باشید
